

Lecture 4: Branching and Looping

Table of contents

1	Objectives	2
2	Branching	2
2.1	Instruction Pointer	3
2.1.1	Relative vs Absolute Offset Address	3
2.2	JMP Instruction	4
2.2.1	Types of Jumps:	5
2.2.2	Example	5
2.3	LOOP Instruction	6
2.3.1	Example	6
2.3.2	More Examples	7
2.3.3	Nested Loop	10
2.4	Testing and Comparing	11
2.4.1	TEST Instruction	11
2.4.2	CMP Instruction	12
2.4.3	Interpreting Flags after CMP	13
2.5	Conditional Jumps	14
2.5.1	Types of Conditional Jump	16
2.5.2	Jump Based on Specific Flag Values	17
2.5.3	Jump Based on CX/ECX/RCX	17

2.5.4	Jump Based on Comparisons of Unsigned Operands . . .	17
2.5.5	Jump Based on Comparisons of Signed Operands . . .	18
2.5.6	Affected Flags	18
2.6	Examples	18
3	Implementing High-Level Flow Structures	20
3.1	If statements	20
4	Switch statements	24
4.1	Loop statements	25
5	Practical notes and common pitfalls	27
6	*Extra instructions: (not included in your final exam)	28
6.1	CMOVcc (Conditional Move)	29
6.1.1	Example	29
6.2	SETcc (Set Byte on Condition)	30
6.2.1	Examples	30

1 Objectives

1. Understand relative and absolute target addresses for control flow
2. Learn the essential x86 instructions for decision-making logic
3. Translate if, switch, and looping constructs from high-level languages into assembly

2 Branching

- **Sequential execution:** By default, the CPU fetches and executes instructions in order.
- A *branch*, or **transfer of control**, is a mechanism that alters the normal execution order.
- There are two basic types of transfer:

1. **Unconditional transfer**, in which a transfer is occurred unconditionally.
2. **Conditional transfer**, in which a transfer is occurred based on a certain condition.

2.1 Instruction Pointer

- The **instruction pointer** (EIP in 32-bit mode, RIP in 64-bit mode) holds the address of the next instruction to be executed.
- Certain machine instructions update EIP/RIP, and cause the program to *branch* to a new location. These instructions are JMP, Jcc, LOOP, CALL, RET, or IRET.
- The general syntax of these instructions is

Syntax

<instruction> <target>

Where <target> is an **offset address**.

2.1.1 Relative vs Absolute Offset Address

- The EIP/RIP can be updated based on a target address.
- The target address, within the instruction stream, can be **relative offset** or **absolute offset**
- A **relative offset** is a signed *displacement* to the current value of the EIP.

$$\text{EIP} = \text{EIP} + \text{Target displacement}$$

- An **absolute offset** is an offset from the base of the code segment (i.e., offset from address 0 of a segment).

$$\text{EIP} = \text{Target Address}$$

- Relative offset can be specified in either of the following ways

Address format	Description
rel8	A constant value in the range from -128 to 127 bytes.
rel16	A relative address within the same code segment.
rel32	A relative address within the same code segment.

Note

A relative offset (rel8, rel16, or rel32) is generally specified as a **label** in assembly code, but at the machine code level, it is encoded as a signed 8-, 16-, or 32-bit immediate value.

- Absolute offset can be specified as an address in general-purpose register or memory.

2.2 JMP Instruction

Syntax

```
JMP    <target>
```

- The JMP instruction causes an unconditional transfer to <target>.
- The <target> operand specifies the address of the instruction being jumped to. This operand can be an immediate value, a general-purpose register, or a memory location.
- Thus, the JMP instruction takes the following types:

JMP	rel8	; relative address
JMP	rel16	; relative address (not supported in 64-b)
JMP	rel32	; relative address
JMP	reg/mem16	; absolute address (not supported in 64-b)
JMP	reg/mem32	; absolute address (not supported in 64-b)

- In this course, we only consider

- JMP rel8,
- JMP rel16, and
- JMP rel32.

2.2.1 Types of Jumps:

1. **Near Jump**, which is a jump to instruction within the current code segment (intra-segment jump). Here the displacement is either rel16, rel32, reg/mem16, reg/mem32.
2. **Short jump**, which is a near jump where the jump is limited to -128 to +127 from the current EIP value. Here the displacement is rel8.
3. **Far Jump**: A jump to an instruction located in a different segment than the current code segment but at the same privilege level (inter-segment jump) (Not covered here).

Note

There is no difference in the coding for *short* and *near* jumps. The assembler automatically uses a short jump if the displacement is within the small range in order to generate more compact code; otherwise, it uses near jump.

2.2.2 Example

```

    mov    ax, 0
top:  mov    bx, 5      ; <--- a label
      add    bx, 1
      ; ...
      jmp    top      ; <--- go back to mov bx, 5

```

2.3 LOOP Instruction

Syntax

```
LOOP    <target>
```



Rule

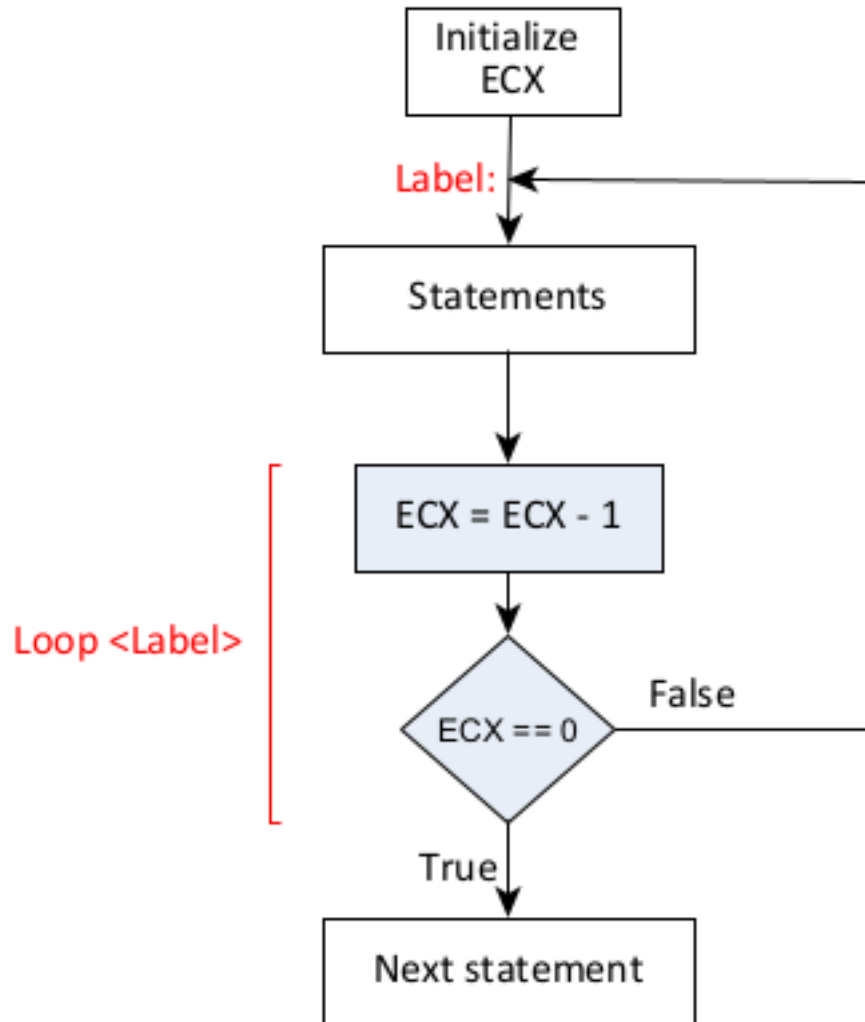
- The target operand must be rel8.
- The loop instruction repeats a block of statements a specific number of times (determined by ECX register in 32-bit mode).
- ECX is automatically used as a counter and is decremented each time the loop repeats.
- LOOP instruction jumps to <target> if ECX != 0.

2.3.1 Example

```

    mov    ax, 1      ; set ax to 1
    mov    ecx, 5     ; repeat 5 times
top:  add    ax, ax    ; the loop body
      loop  top      ; (1) ecx-- (2) if (ecx!=0) jump to top

```



2.3.2 More Examples

Example 2.1. Write an assembly program to compute the sum of the following series

$$S = 1 + 2 + 4 + 8 + \dots + 1024$$

Solution:

Here, we can re-write the series as follows:

$$S = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{10} = \sum_{k=0}^{10} 2^k$$

Therefore, we can set ECX to 10. However, setting ECX to 10 implies that the loop starts from 10 and ends at 1. This also implies that 2^0 won't be added. To handle it, we can add 2^0 when we initialize the sum S .

```

section .data
varS:  dd      0

section .text
_main:
    mov     eax, 1          ; first element is (2^0) = 1
    add     [varS], eax     ; add first element to varS
    mov     ecx, 10        ; set loop counter to 10
top:
    add     eax, eax        ; set eax = 2 * eax
    add     [varS], eax     ; add eax to varS
    loop    top
ret

```

Performance Note

In the previous program, a memory location is accessed during every iteration. If the number of iterations is quite high, then we have too many memory accesses.

Most modern CPUs include **cache memory**, which is a smaller but faster type of memory located close to the processor core.

When the variable `varS` is accessed for the first time, its value is fetched from main memory (RAM) into the cache - a situation known as a **cache miss**. On subsequent memory accesses, the CPU retrieves the value directly from the cache (**cache hit**) instead of RAM. This reuse of recently accessed data is referred to as **temporal locality**. Using cache can dramatically improve the program's execution time.

To make the program even faster, we can store the sum in a CPU register

while the loop is running, and only write the result back to `varS` once the loop is finished (see the “alternative solution” below). This way, we avoid many unnecessary memory accesses and the program runs more quickly. *These examples show the real beauty of assembly language: we have the power to control exactly how the program uses registers and memory. Unlike most high-level language, assembly gives us the freedom to optimize our code step by step and make it as efficient as possible.*

Alternative solution:

*; This solution accesses sum in EBX register while loop
; is running, and only write the result back to varS
; once the loop is finished.*

```
section .bss
varS:  resd    1

section .text
_main:
    ; we store the sum in EBX register
    mov     eax, 1          ; first elm which is 1
    mov     ebx, eax        ; set sum (ebx) to 1
    mov     ecx, 10         ; set loop counter to 10
top:
    add     eax, eax        ; set eax = 2 * eax
    add     ebx, eax        ; add eax to sum (ebx)
    loop    top

    mov     [varS], ebx     ; store the result

    ret
```

Example 2.2. Suppose you have the following signed 16-bit integer array hard-coded in your program:

```
arrX[] = {13, 22, 18, 10, 25}
```

Write an assembly program to store the above array in the data section and then calculate the sum of its elements. Store the sum in `sum` variable.

Solution:

```
section .data
arrX:  dw      13, 22, 18, 10, 25
ARRSIZ equ    ($ - arrX) / 2
sum:   dw      0

section .text
_main:
    mov     edi, arrX      ; edi points to 1st element
    mov     ecx, ARRSIZ    ; set counter
    mov     ax, 0          ; set sum to 0
L1:
    add     ax, [edi]       ; load next elm and add it to sum
    add     edi, 2          ; set edi to point to next elm
    loop    L1             ; loop

    mov     [sum], ax       ; store sum
    ret
```

2.3.3 Nested Loop

- How to implement the following nested loop:

```
for (int i=100; i != 0; i--)
    for (int j=50; j != 0; j--)
        ...
```

in assembly language?

- **Challenge:** LOOP instruction has only *one counter*, which is ECX.
- To solve this challenge, we must preserve the outer loop counter before executing the inner loop, and restore the outer loop counter before executing the outer's LOOP instruction. Here is a sample solution:

```

        section .bss
count:   resd    1           ; to store outer loop counter

        section .text

        ; ...

        mov     ecx, 100    ; # of iterations for outer loop
OUTER:
        mov     [count], ecx ; preserve outer loop counter
        mov     ecx, 50     ; # of iterations for inner loop
INNER:
        ; ...

        loop    INNER
        mov     ecx, [count] ; restore outer loop counter
        loop    OUTER

```

- In future lecture, we can enhance this solution by using **run-time stack**.

2.4 Testing and Comparing

2.4.1 TEST Instruction

Syntax

```
TEST    <destination>, <source>
```

- TEST instruction performs an **implied** AND operation of two operands and set the SF, ZF, and PF flags according to the results. It is implied because it won't store the result in the destination operand
- The flags can then be tested by the conditional jump or loop instruction. We will elaborate more in the next section.

- TEST instruction differs from the AND instruction in that it does not alter either of the operands

Rules

- Both operands must be of the same size
 - Both operands cannot be memory locations
 - Destination operand cannot be immediate
- The TEST instruction is often more efficient than using AND and CMP for simple checks. It is commonly used in practice for tasks such as:

1. Checking if a number is zero:

; EAX contains the number

```
test    eax, eax    ; ZF = 1 if EAX is zero; else ZF = 0
```

2. Checking if a number is negative

; EAX contains the number

```
test    eax, eax    ; SF = 1 if EAX is negative; else SF
```

2.4.2 CMP Instruction

Syntax

```
CMP    <destination>, <source>
```

- CMP instruction is used to compare between two integral values (integers or characters).
- It performs an **implied** subtraction of a source operand from a destination operand.
- It affects ALL status flags according to the result. Operands are not modified.

- The rules applied to CMP are the same rules applied to TEST.

2.4.3 Interpreting Flags after CMP

Through these flags, we can compare the two operands:

- **Unsigned interpretation:**

ZF	CF	Comparison Result
0	0	destination > source
0	1	destination < source
1	*	destination = source

- **Singed interpretation:**

ZF	SF	OF	Comparison Result
0	0	0	destination > source
0	0	1	destination < source
0	1	0	destination < source
0	1	1	destination > source
1	*	*	destination = source

- CMP is a valuable tool for creating conditional logic structures. When you follow CMP with a conditional jump instruction, you can implement any flow control statement (such as if, if else, switch case, while, do while).
- Example:

```

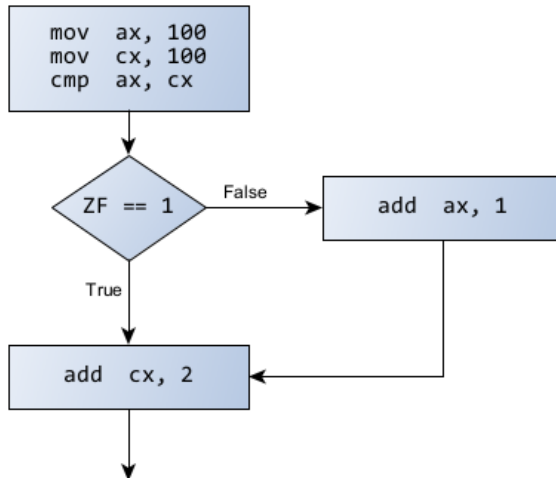
mov    ax, 100
mov    cx, 100
cmp    ax, cx    ; ZF = 1

jz     skip      ; Jump if zero (Discussed next section)

```

```
        add    ax, 1
skip:
        add    cx, 2
```

- The flow control of the previous code is shown below:



2.5 Conditional Jumps

Syntax

```
Jcc    <target>
```

- The Jcc instruction causes a **conditional transfer** to a target address, which can be specified relatively or absolutely.
- The **condition code** (cc) (that comes after letter J) indicates the condition being tested for. Intel microarchitecture has a rich set of condition codes, such as JZ (Jump if ZF = 1), JNZ (Jump if ZF = 0), JC (Jump if CF is set), JNC (Jump if CF is not set), and so on.
- The Jcc instruction supports near jump only.
- **Branching:** If condition (cc) is true, *jump* to a destination label. Otherwise, execute the next instruction.

Label:

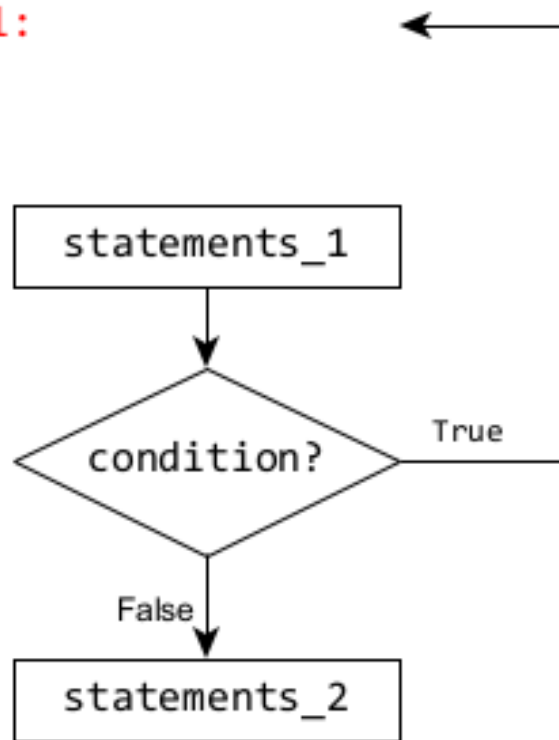


Figure 1: Backward Reference

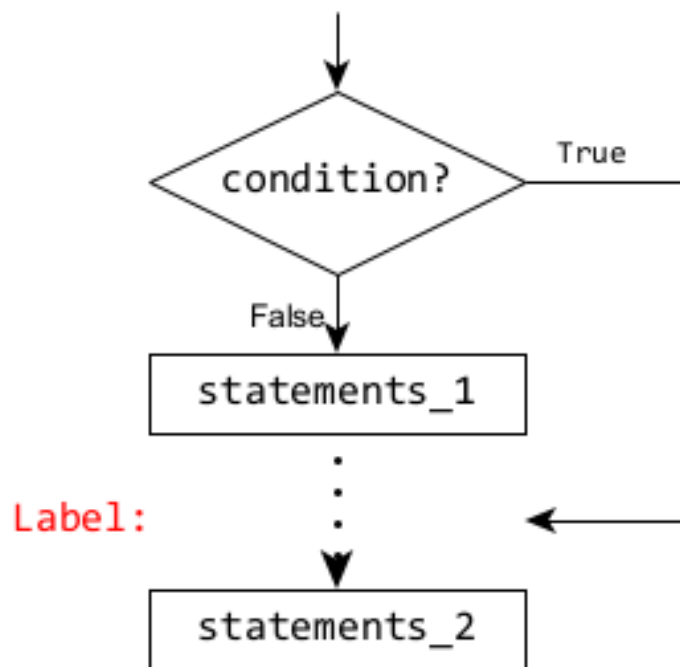


Figure 2: Forward Reference

2.5.1 Types of Conditional Jump

- As mentioned, the x86 instruction set has a large number of conditional jump instructions. They are able to compare signed and unsigned integers and perform actions based on the values of individual CPU flags.
- The conditional jump instructions can be divided into four groups:
 1. Jumps based on specific flag values
 2. Jumps based on the value of RCX, ECX, or CX
 3. Jumps based on comparisons of unsigned operands
 4. Jumps based on comparisons of signed operands

2.5.2 Jump Based on Specific Flag Values

Mnemonic	Meaning	Condition
JZ / JE	Jump if zero / equal	ZF = 1
JNZ / JNE	Jump if not zero / not equal	ZF = 0
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JS	Jump if sign	SF = 1
JNS	Jump if not sign	SF = 0
JP / JPE	Jump if parity even	PF = 1
JNP / JPO	Jump if parity odd	PF = 0

2.5.3 Jump Based on CX/ECX/RCX

Mnemonic	Meaning
JCXZ	Jump if CX = 0
JECXZ	Jump if ECX = 0
JRCXZ	Jump if RCX = 0 (64-bit mode)

2.5.4 Jump Based on Comparisons of Unsigned Operands

Mnemonic	Meaning	Condition
JA	Jump if above	CF = 0 && ZF = 0
JAЕ	Jump if above or equal	CF = 0
JB	Jump if below	CF = 1 && ZF = 0
JBE	Jump if below or equal	CF = 1 ZF = 1

Additional instructions:

Mnemonic	Meaning	Equivalent to
JNBE	Jump if not below of equal	JA
JNB	Jump if not below	JAЕ
JNAE	Jump if not above or equal	JB
JNA	Jump if not above	JNB

2.5.5 Jump Based on Comparisons of Signed Operands

Mnemonic	Meaning	Condition
JG	Jump if greater	ZF=0 && SF=OF
JGE	Jump if greater or equal	SF=OF
JL	Jump if less	SF != OF
JLE	Jump if less or equal	ZF=1 SF != OF

Additional instructions:

Mnemonic	Meaning	Equivalent to
JNLE	Jump if not less or equal	JG
JNL	Jump if not less	JGE
JNGE	Jump if not greater or equal	JL
JNG	Jump if not greater	JLE

2.5.6 Affected Flags

- Conditional jump instructions (Jcc) do not modify flags.

2.6 Examples

Example 2.3.

```

test    eax, eax      ; check if eax is zero/nonzero
jz      is_zero       ; jump if eax is zero
jnz     is_nonzero    ; jump if eax is nonzero

```

Example 2.4.

```

test    eax, 1        ; check if eax is even/odd
jz      even          ; ZF=1 if eax & 1 == 0
jnz     odd           ; otherwise

```

Example 2.5.

```

test    eax, 10000b   ; check fifth bit
jnz     top           ; jump if fifth bit is set

```

Example 2.6.

```

mov     edx, -1       ; edx <-- -1
cmp     edx, 0
jg      L5            ; jump if EDX > 0 (false) => no jump
jle     L6            ; jump if EDX <= 0 (true) => jump to L6

```

Example 2.7.

```

mov     bx, 35        ; bx <-- 35
cmp     bx, 32
jb      L4            ; jump if BX < 32 (false) => no jump
jae     L3            ; jump if BX >= 32 (true) => jump to L3

```

Example 2.8.

```

    ; repeat 10 times
    ; for (ecx = 0; ecx < 10; ecx++) <loop statement>

    mov    ecx, 0
top:

    ; <loop statement>

    inc    ecx
    cmp    ecx, 10
    jb     top    ; jump if ECX < 10

```

3 Implementing High-Level Flow Structures

In this section, you will learn how to translate:

- if statement
- if-else statement
- switch statement
- for loop
- while loop
- do while loop

into assembly language.

3.1 If statements

Example 3.1. Translate the following if statement into assembly language. Assume the variable `x` is unsigned `int`.

```
if (x > 2000)
    x = x/2;
```

Solution:

```
    mov     eax, [x]        ; eax <-- x
    cmp     eax, 2000
    jbe     ENDIF          ; jump if eax <= 2000 to ENDIF
    xor     edx, edx        ; set high to zero for unsigned div
    mov     ebx, 2          ; divisor is 2
    div     ebx             ; eax = eax / 2
    mov     [x], eax        ; x <-- eax
ENDIF:
```

Example 3.2. Translate the following if statement, assuming the variables are declared as int:

```
if (value < 10)
    smallCount++;
else
    largeCount++;
```

Solution:

```
    mov     eax, [value]    ; eax <-- value
    cmp     eax, 10
    jnl     ELSE           ; jump if eax >= 10 to ELSE
    inc     DWORD [smallCount] ; smallCount++
    jmp     ENDIF          ; jump to ENDIF
ELSE:
    inc     DWORD [largeCount] ; largeCount++
ENDIF:
```

Example 3.3. Translate the following statement (all variables are declared as int):

```
if ( (total >= 100) && (count = 10) )
    total += value;
```

Solution:

```
mov    eax, [total]        ; eax <-- total
cmp    eax, 100
jl     ENDIF               ; jump if eax < 100 to ENDIF
cmp    DWORD [count], 10
jnz    ENDIF               ; jump if count != 10 to ENDIF

add    eax, [value]        ; eax += value
mov    [total], eax        ; total <-- value
ENDIF:
```

Example 3.4. Translate the following statement (all variables are declared as int):

```
if ( (total >= 100) || (count = 10) )
    total += value;
```

Solution:

```
mov    eax, [total]        ; eax <-- total
cmp    eax, 100
jge    THEN                ; jump if eax >= 100 to THEN
cmp    DWORD [count], 10
jnz    ENDIF               ; jump if count != 0 to ENDIF
THEN:
add    eax, [value]
mov    [total], eax
ENDIF:
```

Example 3.5. Find the smallest of three integers:

```
if (a <= b and a <= c)
    smallest = a;
else if (b <= c)
    smallest = b;
else
    smallest = c;
```

Solution:

```
    mov     eax, [a]           ; eax <-- a
    mov     ebx, [b]           ; ebx <-- b
    mov     ecx, [c]           ; ecx <-- c
    cmp     eax, ebx
    jg      ELSE1              ; jump if eax > ebx to ELSE1
    cmp     eax, ecx
    jg      ELSE1              ; jump if eax > ecx to ELSE1

    mov     [smallest], eax
    jmp     ENDIF              ; jump to ENDIF

ELSE1:
    cmp     ebx, ecx
    jg      ELSE2              ; jump if ebx > ecx to ELSE2

    mov     [smallest], ebx
    jmp     ENDIF              ; jump to ENDIF

ELSE2:
    mov     [smallest], ecx

ENDIF:
```

4 Switch statements

Example 4.1. Translate the following switch statement:

```
switch (x) {  
  case 0:  
    y = 1;  
    break;  
  case 2:  
    y = 10;  
    break;  
  case 4:  
    y = 100;  
    break;  
  default:  
    y = 0;  
}
```

Solution:

```
    mov    eax, [x]          ; eax <-- x  
    cmp    eax, 0  
    jnq    CASE2             ; jump if x != 0 to CASE2  
  
    mov    ebx, 1            ; ebx <-- 1  
    jmp    ENDCASE           ; jump to ENDCASE  
CASE2:  
    cmp    eax, 2  
    jnq    CASE3             ; jump if x != 2 to CASE3  
  
    mov    ebx, 10           ; ebx <-- 10  
    jmp    ENDCASE           ; jump to ENDCASE  
CASE3:  
    cmp    eax, 4  
    jnq    CASE4             ; jump if x != 4 to CASE4
```

```

        mov     ebx, 100          ; ebx <-- 100
        jmp     ENDCASE          ; jump to ENDCASE
CASE4:
        mov     ebx, 0           ; ebx <-- 0 (default)
ENDCASE:
        mov     [y], ebx         ; y <-- ebx

```

4.1 Loop statements

Example 4.2. Translate the following for loop:

```

sum = 0;
for (int i=1; i<100; i+=2)
    sum += i;

```

Solution:

```

        xor     eax, eax         ; set sum to zero
        mov     ecx, 1          ; set index to 1
top:
        add     eax, ecx         ; sum += ecx
        add     ecx, 2          ; increment index by 2
        cmp     ecx, 100
        jl      top             ; loop if ecx < 100
        mov     [sum], eax       ; store sum

```

Example 4.3. Translate the following while loop:

```

d1 = 100;
d2 = 0;
t = 0;
while (d2 <= d1) {
    t = t + 1;
    d1 = d1 - 3;
}

```

```

    d2 = 2*t - 1;
}

```

Solution:

```

    mov     eax, 100      ; eax <-- 100
    mov     ebx, 0        ; ebx <-- 0
    mov     ecx, 0;       ; ecx <-- 0
WHILE:
    cmp     ebx, eax
    jg      ENDWHILE      ; jump if b > a to ENDWHILE
    ; loop body
    inc     ecx           ; t++
    sub     eax, 3        ; d1 -= 3
    imul    ebx, ecx, 2    ; d2 = 2*t
    sub     ebx, 1        ; d2 -= 1
    jmp     WHILE
ENDWHILE
    mov     [d1], eax     ; d1 <-- eax
    mov     [d2], ebx     ; d2 <-- ebx
    mov     [t], ecx      ; t <-- ecx

```

Example 4.4. Translate the following do while loop:

```

count = 1;
sum = 0;
do {
    sum = sum / 2 + count * 2
    count++;
} while (sum < 1000 or count <= 100);

```

Solution:

```

    mov     ecx, 1          ; set countn to 1
    mov     eax, 0          ; set sum to 0
DOWHILE:
    cdq                     ; sign extend eax
    mov     ebx, 2          ; divisor
    idiv    ebx             ; eax = eax / 2
    add     eax, ecx         ; eax += count
    add     eax, ecx         ; eax += count
    inc     ecx             ; count ++

    cmp     eax, 1000
    jl      DOWHILE         ; jump if sum < 1000 to DOWHILE
    cmp     ecx, 100
    jle     DOWHILE         ; jump if count <= 100 to DOWHILE
    ; out of do while loop
    mov     [count], ecx    ; countn <-- ecx
    mov     [sum], eax      ; sum <-- eax

```

5 Practical notes and common pitfalls

- **Signed vs. unsigned:** Choose the correct conditional jumps (JG/JL/... for signed, JA/JB/... for unsigned) consistent with your data.
- **Division setup:**
 - **Unsigned div:** Clear high part (xor edx, edx) before div on 32-bit.
 - **Signed idiv:** Use cdq to sign-extend EAX into EDX:EAX.
- **Preserving counters:** For nested LOOPS, save the outer ECX (e.g., push ecx / inner loop / pop ecx) or use explicit counters with cmp/jcc.
- **Short vs. near jumps:** Let the assembler choose; write clear labels and let it optimize encodings.

- **Use TEST for bit checks:** It's faster and clearer than AND when you only need flags.
- Multi-condition short-circuit (AND)

```

; if (a > b && c != 0) { ... }
mov    eax, [a]
cmp    eax, [b]
jle    ENDIF
mov    ecx, [c]
test   ecx, ecx
jz     ENDIF
THEN:
; body ...
ENDIF:

```

- Multi-condition short-circuit (OR)

```

; if (x == 0 || y < 0) { ... }
mov    eax, [x]
test   eax, eax
jz     THEN
mov    ebx, [y]
test   ebx, ebx
jl     THEN
jmp    ENDIF
THEN:
; body ...
ENDIF:

```

6 *Extra instructions: (not included in your final exam)

- Intel x86/x86-64 has a rich set of conditional instructions beyond just Jcc (jump on condition). The ISA leverages the FLAGS register (especially ZF, SF, CF, OF, PF) to make decisions, and Intel designed multiple instruction

families that act based on conditions. Two examples of those families are CMOV_{cc} and SET_{cc}.

6.1 CMOV_{cc} (Conditional Move)

- This family performs a move operation if the status flags in the EFLAGS are in specified state (or condition).
- The condition codes (cc) are similar to the condition codes of J_{cc}.
- This implies you can move data without branching. Hence, improve the performance of your program.

6.1.1 Example

- `a = max(a, b);`

```
mov    eax, [a]    ; eax <-- a
mov    ebx, [b]    ; ebx <-- b
cmp    eax, ebx
cmovl  eax, ebx    ; move if (a < b)
mov    [a], eax    ; a = max(a, b)
```

- `a = |a|`

```
mov    eax, [a]
mov    ebx, eax    ; Copy EAX into EBX
neg    ebx        ; EBX = -EAX
test   eax, eax    ; Set flags based on EAX (check if negative)
cmovl  eax, ebx    ; If (SF OF) => EAX < 0, so EAX = EBX
mov    [a], eax
```

6.2 SETcc (Set Byte on Condition)

Syntax

```
SETcc <destination>
```

- Set the destination to 1 if the condition is true. Otherwise, set to 0.
- The destination must be reg/mem8

6.2.1 Examples

```
b = (a < 0)? 1 : 0;
```

```
mov    eax, [a]
test   eax, eax
sets   bl
movzx  ebx, bl
mov    [b], ebx
```

```
b = (a % 2 == 0) 1 : 0;
```

```
mov    eax, [a]
test   eax, 1
sete   bl
movzx  ebx, bl
mov    [b], ebx
```